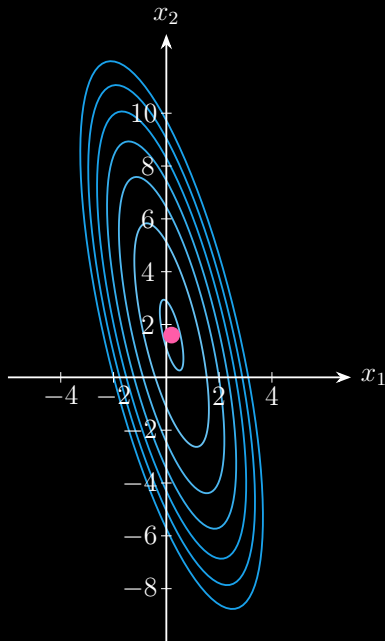


Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

with

$$\mathbf{A} = \begin{bmatrix} 9 & 2 \\ 2 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 5 \\ 2 \end{bmatrix}$$



Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

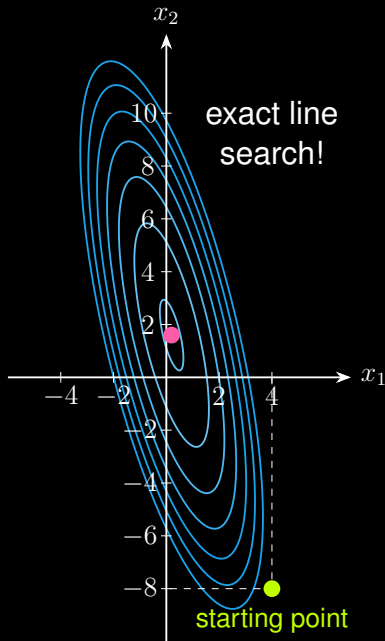
Steepest gradient descent:

$$\mathbf{g}_k = \mathbf{A} \mathbf{x}_k - \mathbf{b} \quad (\text{gradient})$$

$$\alpha_k = \frac{\mathbf{g}_k^\top \mathbf{g}_k}{\mathbf{g}_k^\top \mathbf{A} \mathbf{g}_k} \quad (\text{step size})$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k \quad (\text{decision vector})$$

with an initial point $\mathbf{x}_0$.



Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

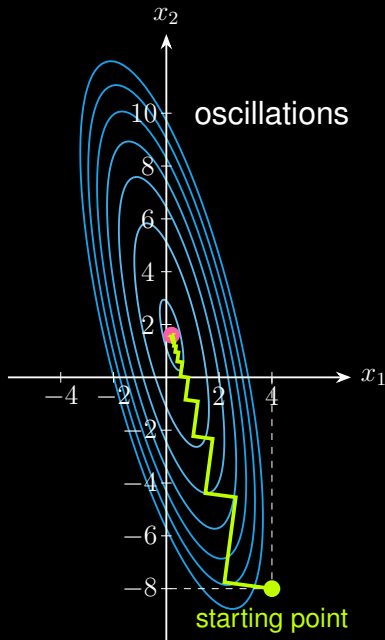
Steepest gradient descent:

$$\mathbf{g}_k = \mathbf{A} \mathbf{x}_k - \mathbf{b} \quad (\text{gradient})$$

$$\alpha_k = \frac{\mathbf{g}_k^\top \mathbf{g}_k}{\mathbf{g}_k^\top \mathbf{A} \mathbf{g}_k} \quad (\text{step size})$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k \quad (\text{decision vector})$$

with an initial point $\mathbf{x}_0$.



Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

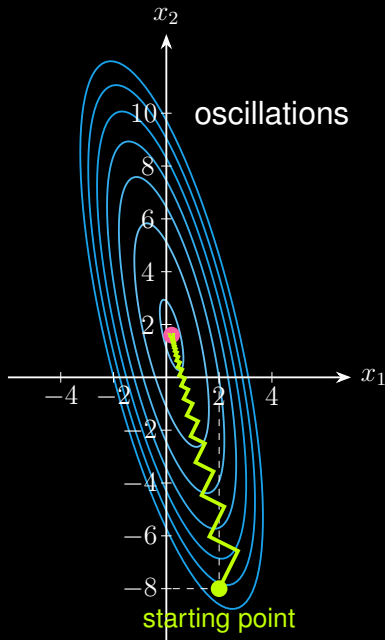
Steepest gradient descent:

$$\mathbf{g}_k = \mathbf{A} \mathbf{x}_k - \mathbf{b} \quad (\text{gradient})$$

$$\alpha_k = \frac{\mathbf{g}_k^\top \mathbf{g}_k}{\mathbf{g}_k^\top \mathbf{A} \mathbf{g}_k} \quad (\text{step size})$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k \quad (\text{decision vector})$$

with an initial point $\mathbf{x}_0$.



Positive Definite Matrix $A = \begin{bmatrix} 9 & 2 \\ 2 & 1 \end{bmatrix}$

Eigenvalue decomposition:

$$A = \lambda_1 \mathbf{u}_1 \mathbf{u}_1^\top + \lambda_2 \mathbf{u}_2 \mathbf{u}_2^\top$$

with eigenvalues:

$$\lambda_1 = 9.47 \quad \lambda_2 = 0.53$$

and condition number:

$$\kappa = \frac{\lambda_1}{\lambda_2} = 17.94$$

Positive Definite Matrix $A = \begin{bmatrix} 9 & 2 \\ 2 & 1 \end{bmatrix}$

Eigenvalue decomposition:

$$A = \lambda_1 \mathbf{u}_1 \mathbf{u}_1^\top + \lambda_2 \mathbf{u}_2 \mathbf{u}_2^\top$$

with eigenvalues:

$$\lambda_1 = 9.47 \quad \lambda_2 = 0.53$$

and condition number:

$$\kappa = \frac{\lambda_1}{\lambda_2} = 17.94$$

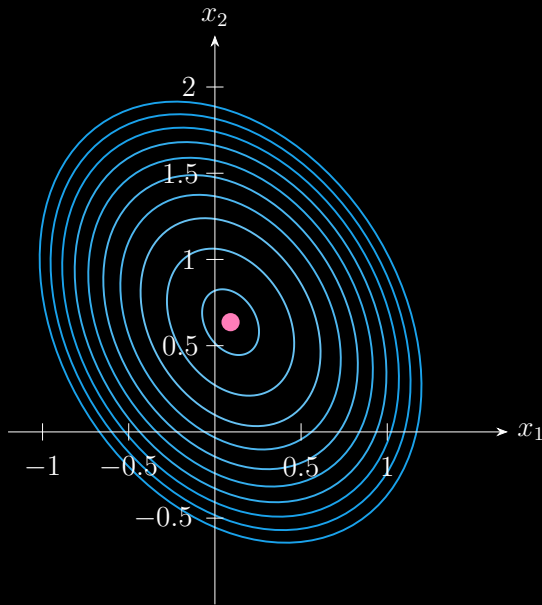
```
1 import numpy as np
2
3 A = np.array([[9, 2], [2, 1]])
4 eig_val, eig_vec = np.linalg.eig(A)
5
6 print('lambda_1 = ', eig_val[0])
7 print('lambda_2 = ', eig_val[1])
8 print('u_1 = ', eig_vec[:, 0])
9 print('u_2 = ', eig_vec[:, 1])
10 print('condition number = ',
11       eig_val.max() / eig_val.min())
```

Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

with

$$\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

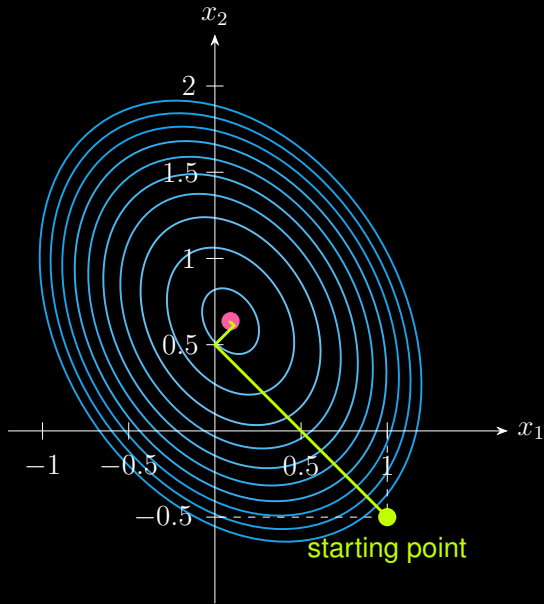


Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

with

$$\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

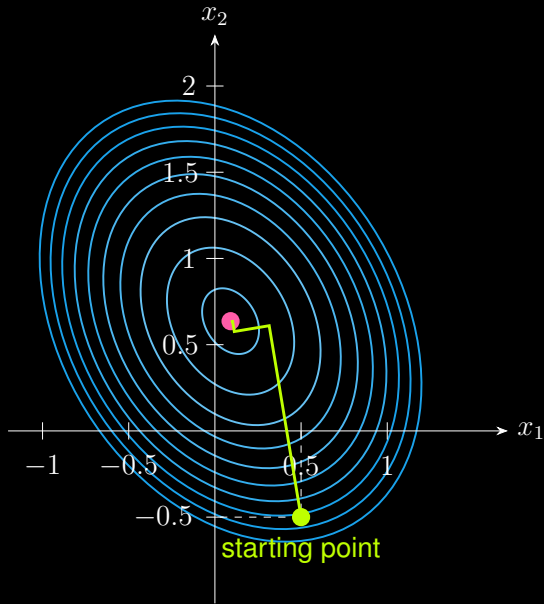


Quadratic optimization:

$$\min_x \quad \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

with

$$\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$



Positive Definite Matrix $A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$

Eigenvalue decomposition:

$$A = \lambda_1 \mathbf{u}_1 \mathbf{u}_1^\top + \lambda_2 \mathbf{u}_2 \mathbf{u}_2^\top$$

with eigenvalues:

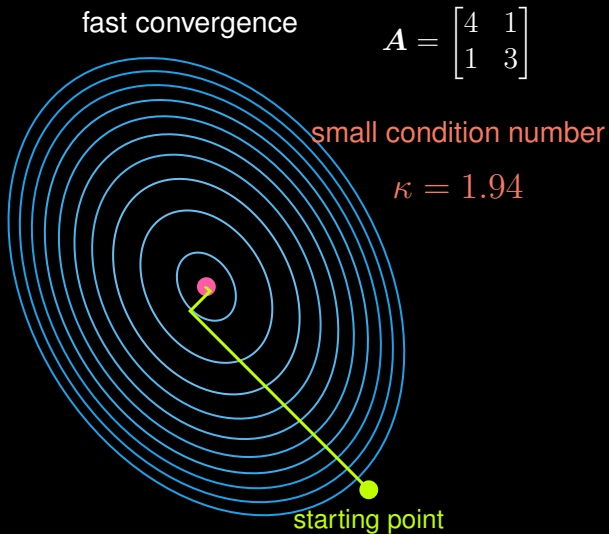
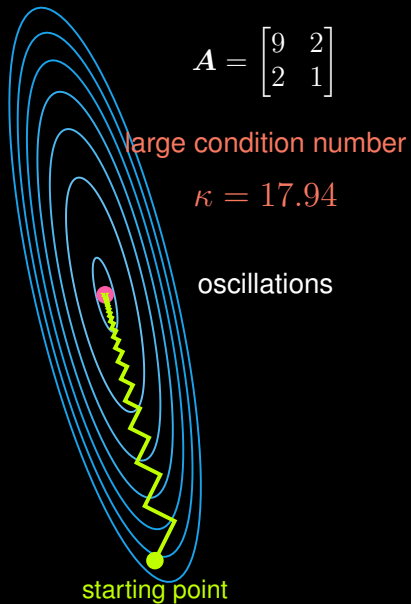
$$\lambda_1 = 4.62 \quad \lambda_2 = 2.38$$

and condition number:

$$\kappa = \frac{\lambda_1}{\lambda_2} = 1.94$$

```
1 import numpy as np
2
3 A = np.array([[4, 1], [1, 3]])
4 eig_val, eig_vec = np.linalg.eig(A)
5
6 print('lambda_1 = ', eig_val[0])
7 print('lambda_2 = ', eig_val[1])
8 print('u_1 = ', eig_vec[:, 0])
9 print('u_2 = ', eig_vec[:, 1])
10 print('condition number = ',
11       eig_val.max() / eig_val.min())
```

Steepest Gradient Descent



Thanks for your attention!

About me:

🏠 Homepage: <https://xinychen.github.io>

✉ How to reach me: chenxy346@gmail.com